SANDIP UNIVERSITY

Experiment No – 01

Title: Python implementation to convert foreign language to English language using machine translation (German to English)

Theory:

This experiment demonstrates Machine Translation (MT), a key task in Natural Language Processing (NLP). Machine translation is the automatic conversion of text from one human language into another while maintaining its meaning. Modern translation systems like Google Translate are based on advanced neural network models, which use massive multilingual datasets to provide accurate results.

Here, we make use of the deep-translator library, which provides an interface to Google Translate. The experiment is designed to translate German text (source language) into English text (target language).

NLP Concepts Involved:

- **Machine Translation:** Automatically converting a sentence from one language to another.
- **Tokenization:** Breaking down text into smaller units (words or sub-words) before translation.
- **Encoder–Decoder Models:** The German input is encoded into vector form, then decoded into English using neural models.
- **Context Preservation:** Translation is not done word-for-word; instead, the model ensures that the meaning of the whole sentence is preserved.
- **Error Handling in NLP:** Systems must handle unexpected or invalid inputs to prevent incorrect translation or program crashes.

SANDIP UNIVERSITY

How the Code Works:

1. Initialization of Translator

The program begins by creating a translator object, which is set to translate from German (de) to English (en). This sets up the machine translation engine before any input is taken.

2. User Input Handling

The program continuously asks the user to enter text in German. This makes the program interactive and capable of translating multiple sentences without restarting.

3. Exit Condition

If the user enters the word "quit," the program recognizes this as a command to stop execution. It then exits gracefully, showing a closing message.

4. Validation of Input

Before attempting translation, the program checks if the user's input is empty or just spaces. If so, it prompts the user to enter valid text. This prevents unnecessary errors during translation.

5. Translation Process

For valid input, the program sends the German text to the Google Translate API through the deep-translator library. The API internally performs tokenization, encoding, and decoding using neural translation models. The output is an accurate English translation of the input sentence.

6. Display of Output

The translated English text is displayed to the user in a simple and readable format. This allows the user to immediately view the translation result.

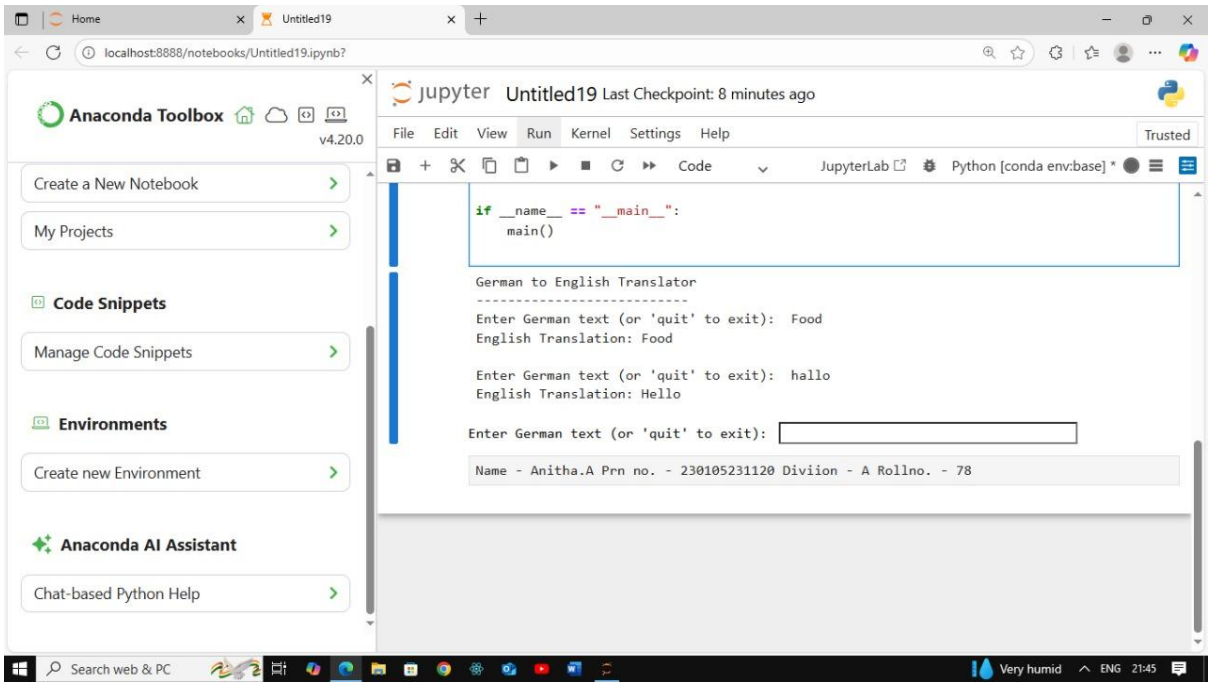
7. Error Handling

If any issue arises (such as loss of internet connection or invalid request), the program catches the error and informs the user instead of crashing. This improves the reliability of the system.

Code:

```
from deep_translator import GoogleTranslator

def main():

print("Enter German text (or 'quit' to exit):")

translator = GoogleTranslator(source='de', target='en')

while True:


text = input("> ")

if text.lower() == 'quit':

print("Exiting.")

break

if not text.strip():

print("Enter valid text.")

continue

try:

translated = translator.translate(text)

print(f"English: {translated}")

except Exception as e:

print(f"Error: {e}")

if _name_ == "_main_":

main()
```

Output:



Conclusion:

This experiment successfully demonstrates the use of machine translation in Natural Language Processing. By integrating the deep-translator library with Python, German text is effectively translated into English in real time. The program combines fundamental programming concepts such as loops, conditions, and exception handling with advanced NLP tasks like tokenization and context-based translation. This highlights how NLP can be applied in real-world applications to remove language barriers and make communication easier across different languages.

SANDIP UNIVERSITY

Experiment No – 02

Title: Explain the process of word and sentence tokenization using the NLTK library in Python

Theory:

In **Natural Language Processing (NLP)**, raw text must be broken down into smaller units before applying advanced operations. This process is known as **tokenization**. Tokenization plays a fundamental role in preparing text for further NLP tasks such as part-of-speech tagging, parsing, information retrieval, and machine translation.

There are two main types of tokenization:

- **Sentence Tokenization:** Splitting a paragraph or text into individual sentences. This is useful when analyzing sentence-level meaning or performing tasks like machine translation and summarization.
- **Word Tokenization:** Breaking down sentences into words, punctuation marks, and other meaningful units. This is essential for tasks like sentiment analysis, keyword extraction, and text classification.

In this experiment, we use the **NLTK (Natural Language Toolkit)** library in Python, which is a widely used library for NLP research and applications. NLTK provides pre-trained tokenizers such as **PunktSentenceTokenizer** for sentence splitting and **TreebankWordTokenizer** for word tokenization.

NLP Concepts Involved:

- **Text Preprocessing:** Tokenization is the first step in preprocessing raw text.
- **Sentence Boundary Detection:** Recognizing where one sentence ends and another begins.
- **Word Segmentation:** Identifying individual words in a sentence, including punctuation.

- **NLTK Toolkit:** A standard Python library that provides tools for text processing, tokenization, tagging, and parsing.

- **Applications:** Tokenization is the foundation for most downstream NLP tasks such as text summarization, translation, and classification.

How the Code Works:

1. Input Text

The program begins with a sample text:
"My name is Anitha Avirneni and I'm studying in Sandip University. My branch is B.Tech CSE AIML."
This paragraph contains two sentences and multiple words.

2. Sentence Tokenization

The program applies **sentence tokenization** using NLTK's pre-trained sentence tokenizer. It detects sentence boundaries (using punctuation and capitalization cues) and splits the text into two sentences:

- Sentence 1: " Sandip University is the state University in Maharastra."
- Sentence 2: "I love Sandip University."

3. Word Tokenization

Next, the program applies **word tokenization**. It breaks down the sentences into individual tokens, which include words, punctuation marks, and abbreviations. For example:

- "Sandip", "University", "is", "the", "'state", "University", "in", "Maharastra", "."
- "I", "love", "Sandip", "University", "."

4. Storage of Tokens

The results are stored separately in lists:

- One list for **sentence tokens** (two items in this case).
- Another list for **word tokens** (all words and punctuation marks).

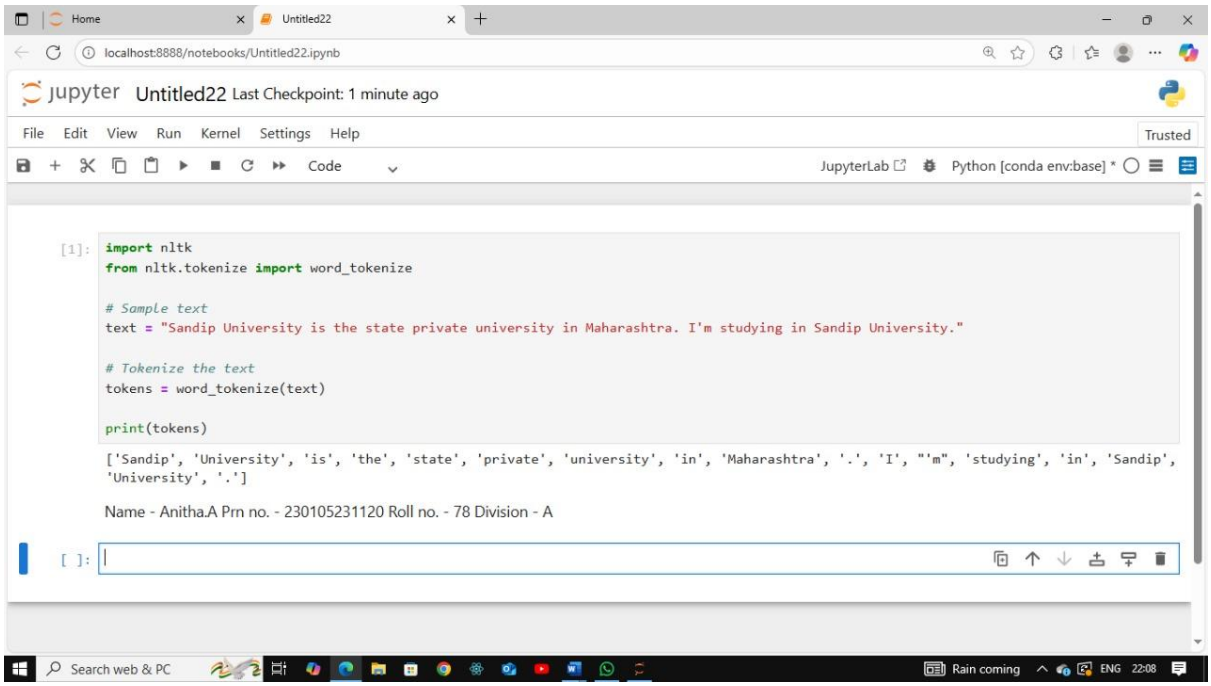
5. Output

Finally, the program prints the sentence tokens and word tokens. This demonstrates how the text is segmented at both the sentence level and the word level.

Code:

```
from nltk import sent_tokenize,word_tokenize

text = 'Sandip University is the state University in Maharastra. I love Sandip University'

sent_tokens = sent_tokenize(text)

sent_tokens

word_tokens = word_tokenize(text)

word_tokens
```

Output:



SANDIP UNIVERSITY

Conclusion:

This experiment demonstrates the process of tokenization, which is the first and most important step in Natural Language Processing. Using the NLTK library, the text was successfully split into sentences and words. Sentence tokenization helps in analyzing sentence-level meaning, while word tokenization enables tasks such as frequency analysis, part-of-speech tagging, and sentiment detection. This experiment highlights the importance of tokenization as the foundation for all further NLP tasks.

SANDIP UNIVERSITY

Experiment No – 03

Title:- Explain the process of stemming using the NLTK library's PorterStemmer, referencing the provided code which demonstrates stemming on a list of words

Theory:

Stemming is a fundamental preprocessing technique in **Natural Language Processing (NLP)**. It is the process of reducing words to their **root form** by removing prefixes or suffixes. The root word, known as the **stem**, may not always be a valid dictionary word, but it represents the common base form of related words.

For example:

- "playing", "played", "plays" → "play"
- "hitting", "hits" → "hit"

The **Porter Stemmer** is one of the most commonly used stemming algorithms. It was developed by Martin Porter in 1980 and works by applying a set of predefined rules to systematically remove common endings like *-ing*, *-ed*, *-s*, etc.

In this experiment, we use the **NLTK library's PorterStemmer** to demonstrate stemming on a sample sentence.

NLP Concepts Involved:

- **Text Preprocessing:** Stemming is an essential step before applying tasks like classification, clustering, or sentiment analysis.
- **Porter Stemmer Algorithm:** A rule-based algorithm that reduces words to their stems.
- **Word Tokenization:** Before stemming, the text must be broken down into individual words.
- **Dimensionality Reduction:** By reducing words to stems, the vocabulary size is minimized, making models faster and more efficient.

- **Difference from Lemmatization:** Unlike lemmatization, stemming does not guarantee that the root form is a valid word; it is faster but less precise.

How the Code Works:

1. Importing Libraries

The program begins by importing NLTK and its PorterStemmer. It also ensures that the **punkt tokenizer package** is available for splitting text into words.

2. Input Text

The sample input text is:
"My name is Anitha Avirneni and my hobbies are Dancing, Reading, Painting."

3. Word Tokenization

The text is first tokenized into individual words. This step produces a list such as:
["My", "name", "is", " *Anitha Avirneni* ", "and", "my", "hobbies", "are", “Dancing”, “,”, “Reading”, “,”, “Painting”, “.”]

4. Applying Porter Stemmer

The Porter Stemmer is applied to each word in the list. It processes every word and reduces it to its root form. For example:

- "Dancing" → “Danc”
- "Reading" → “Read”
- "Painting" → “Paint”

5. Storing the Stemmed Words

The output is stored in a new list containing all the stemmed words. This shows how multiple variations of the same word are reduced to a common base form.

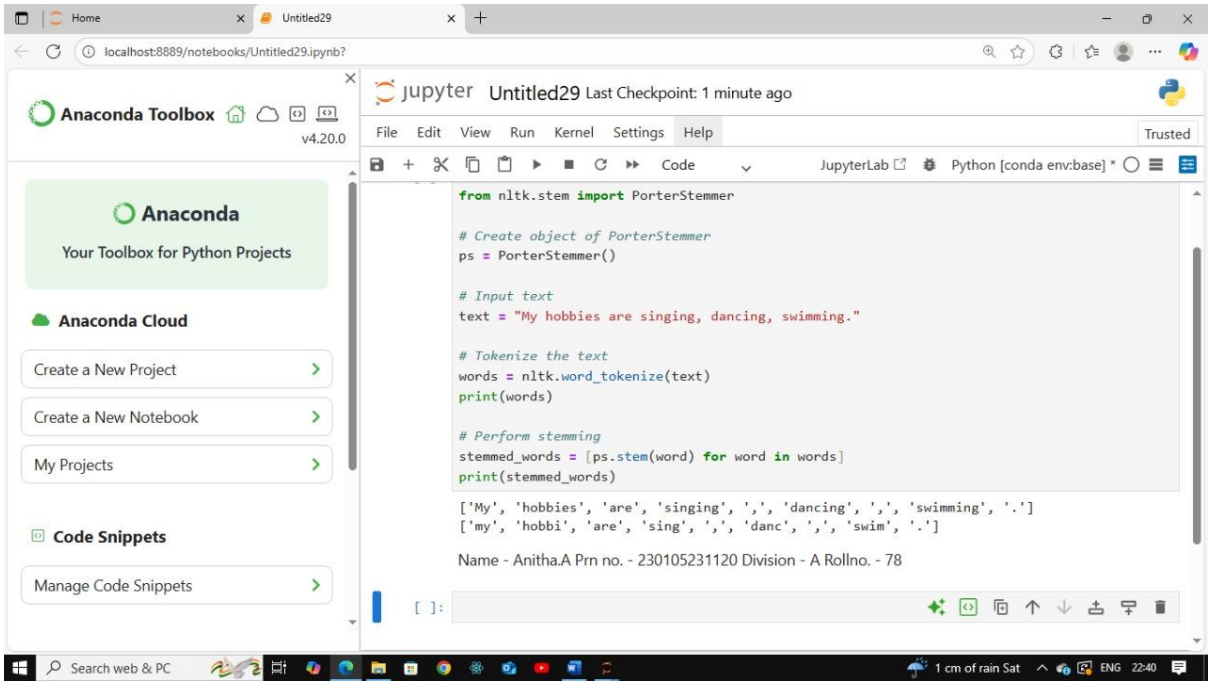
6. Display of Output

Finally, the program prints both the **original tokenized words** and the **stemmed words** side by side, making it easy to compare the changes.

Code:

```
import nltk

nltk.download('punkt')

from nltk.stem import PorterStemmer

ps =PorterStemmer()

text = 'My name is Anitha and my hobbies are hitting gym,playing,swimming,dancing,reading'

word = nltk.word_tokenize(text)

print(word)

stemmed_word = [ps.stem(word) for word in word]

print(stemmed_word)
```

Output:



SANDIP UNIVERSITY

Conclusion:

This experiment demonstrates how stemming works using the **Porter Stemmer** in the NLTK library. By reducing words to their stems, different word forms such as "playing," "played," and "plays" can be treated as the same root "play." This reduces vocabulary size and improves the efficiency of NLP applications. Although stemming may sometimes produce stems that are not real words, it is computationally efficient and widely used in information retrieval, search engines, and text mining tasks.

SANDIP UNIVERSITY

Experiment No – 04

Title: Explain the process of lemmatization using the NLTK library's WordNetLemmatizer

Theory:

Lemmatization is a text normalization technique in **Natural Language Processing (NLP)** that reduces words to their **base or dictionary form**, known as the **lemma**. Unlike stemming, which simply chops off suffixes, lemmatization uses **linguistic knowledge** and considers the **part of speech (POS)** of a word to return valid dictionary words.

For example:

- "running" → "run"
- "went" → "go"
- "files" → "file"

In this experiment, the **WordNet Lemmatizer** from the **NLTK library** is used. WordNet is a large lexical database of English that provides word meanings, synonyms, antonyms, and morphological forms. By combining WordNet with the lemmatizer, we can obtain accurate lemmas.

NLP Concepts Involved:

- **Lemmatization:** Reducing a word to its meaningful base form (lemma).
- **Part-of-Speech (POS) Tagging:** Lemmatization requires POS information (noun, verb, adjective, etc.) to produce correct lemmas. For example, "better" as an adjective lemmatizes to "good."
- **WordNet Database:** A lexical resource used to map words to their lemmas.
- **Difference from Stemming:** Lemmatization always produces valid dictionary words, whereas stemming may generate incomplete or incorrect forms.

- **Applications:** Lemmatization is widely used in search engines, chatbots, text classification, and sentiment analysis where accurate word meaning is essential.

How the Code Works (Conceptual):

1. Importing WordNetLemmatizer

The program begins by importing the WordNet Lemmatizer from NLTK and downloading the required WordNet data. This database is necessary for mapping words to their base forms.

2. Initializing the Lemmatizer

An object of the **WordNetLemmatizer** is created, which provides the functionality for performing lemmatization on words.

3. Input Words

A list of words is defined: *["running", "files", "went", "doing"]*. These words represent different grammatical forms (verbs, plurals, past tense).

4. Lemmatization Process

Each word is passed into the lemmatizer with **part-of-speech set to verb (pos='v')**. This ensures that the lemmatizer treats the words as verbs while generating lemmas. For example:

- "running" → "run"
- "files" → "file"
- "went" → "go"
- "doing" → "do"

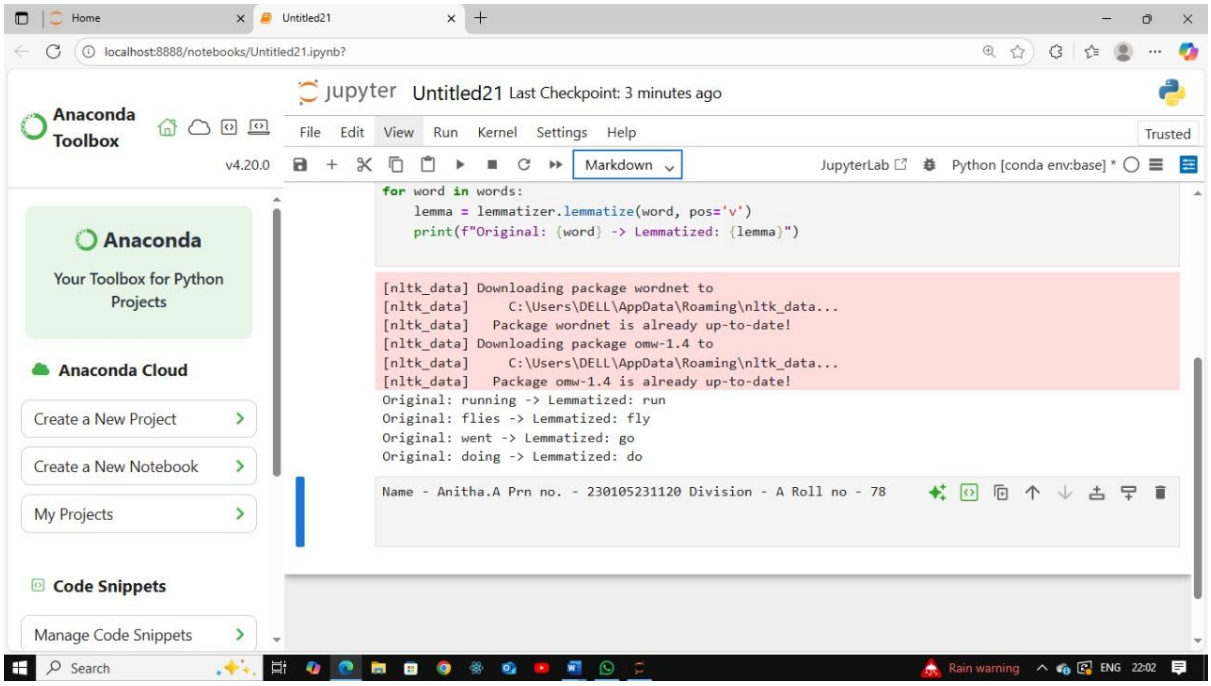
5. Output

The program prints both the original words and their lemmatized forms. This demonstrates how lemmatization transforms different variations of words into a single dictionary root form.

Code:

```
from nltk.stem import WordNetLemmatizer
from nltk.corpus import wordnet
nltk.download('wordnet')
nltk.download('omw-1.4')
lemmatizer = WordNetLemmatizer()
words = ["running", "files", "went", "doing"]

for word in words:
  lemma = lemmatizer.lemmatize(word, pos = 'v')
  print(f"Original: {word}, Lemmatized: {lemma}")
```

Output:



Conclusion:

This experiment demonstrates the process of **lemmatization** using the WordNet Lemmatizer in the NLTK library. By mapping words to their base forms using WordNet and considering part-of-speech tags, lemmatization provides more accurate results than stemming. For example, irregular verbs like "went" are correctly lemmatized to "go." This highlights why lemmatization is preferred in applications where word meaning and grammatical correctness are important, such as semantic analysis, question answering, and text understanding.

SANDIP UNIVERSITY

Experiment No – 05

Title: Explain the process of Part-of-Speech (POS) tagging using the NLTK library in Python

Theory:

Part-of-Speech (POS) tagging is the process of assigning grammatical categories to words in a sentence, such as **noun, verb, adjective, pronoun, preposition**, etc. It is a core task in **Natural Language Processing (NLP)** because knowing the role of words helps in understanding the meaning of sentences.

For example:

- In the sentence *"India is the 7th largest country by the area in the world"*:
    - "India" → Proper Noun (NNP)
    - "is" → Verb (VBZ)
    - "largest" → Adjective (JJS)
    - "country" → Noun (NN)

POS tagging is useful in various NLP applications such as **information extraction, question answering, machine translation, sentiment analysis, and text-to-speech systems**.

The **NLTK library** provides an implementation of POS tagging using the **Averaged Perceptron Tagger**, which is a machine learning–based model trained on large corpora of English text.

NLP Concepts Involved:

- **POS Tagging:** Assigning grammatical roles to each word in a sentence.
- **Tokenization:** The sentence must first be split into individual words before tagging.

- **Statistical Tagger:** NLTK uses the averaged perceptron algorithm, which is a supervised machine learning model, to assign tags.
- **Word Context:** The tagger uses surrounding words to determine the correct part of speech (e.g., "book" as a noun vs "book" as a verb).
- **Applications:** POS tagging helps in parsing, text mining, and building intelligent NLP systems.

How the Code Works:

1. Importing Libraries

The program imports necessary functions from the NLTK library for tokenization and POS tagging. It also downloads the required tokenizer and the averaged perceptron tagger model.

2. Input Text

The given sentence is:
"India is the 7th largest country by the area in the world."

3. Tokenization

The sentence is split into individual words and punctuation marks using the word tokenizer. Example tokens:
["India", "is", "the", "7th", "largest", "country", "by", "the", "area", "in", "the", "world"]

4. Applying POS Tagging

Each token is passed to the POS tagger, which assigns the most likely grammatical category based on learned language rules. For example:

- "India" → NNP (Proper Noun)
- "is" → VBZ (Verb, 3rd person singular present)
- "largest" → JJS (Adjective, superlative)
- "country" → NN (Noun, singular)
- "world" → NN (Noun, singular)

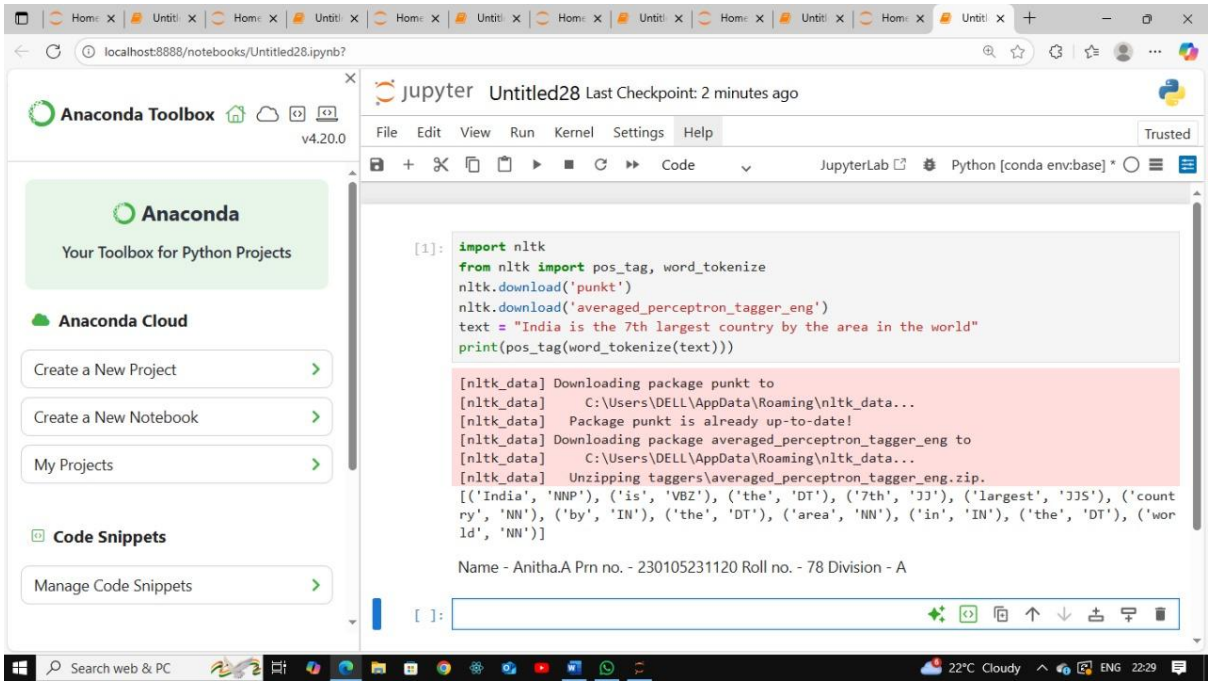
5. Output

The program prints the tokens along with their assigned POS tags as a list of tuples, e.g.: [('India', 'NNP'), ('is', 'VBZ'), ('the', 'DT'), ('7th', 'JJ'), ('largest', 'JJS'), ('country', 'NN'), ('by', 'IN'), ('the', 'DT'), ('area', 'NN'), ('in', 'IN'), ('the', 'DT'), ('world', 'NN')]

Code:

```
import nltk

from nltk import pos_tag, word_tokenize

nltk.download('punkt')

nltk.download('averaged_perceptron_tagger_eng')

text = "India is the 7th largest country by the area in the world"

print(pos_tag(word_tokenize(text)))
```

Output:



SANDIP UNIVERSITY

Conclusion:

This experiment demonstrates the process of **POS tagging** using NLTK's Averaged Perceptron Tagger. By first tokenizing the text and then applying POS tagging, each word in the sentence is assigned a grammatical category. This is a crucial step in NLP pipelines, as it provides structural and syntactic information about the text. POS tagging is widely used in applications such as text summarization, machine translation, and question answering systems, making it a fundamental tool for understanding natural language.